

C++ İLE VERİ YAPILARI DERSİNE HOŞGELDİNİZ

Dr. Öğr. Üyesi Fatma AKALIN

Bilgisayar Bilimleri Dünyası

Bilgisayar yazılımı denince akla geniş bir yelpaze gelir. En basitinden 2 dereceden denklemin çözümü, en karmaşığından büyükçe bir şirketini kurumsal otomasyonu veya bir işletim sistemi tasarımı birer yazılım sürecidir.

Yazılım, genel olarak bir işin program kodları üretilerek yapıldığını belirtir. Donanım dışında kalan ve tasarımcının kodlama yaparak istenilen bir işin veya görevin yerine getirilmesi için oluşturduğu programlar kümesidir

Program belirli bir amaç için tasarlanmış koddan oluşan bir yazılım parçasıdır. Kendi içerisinde bir bütün olan ve belirli bir işi yerine getiren algoritmik bir ifadedir. Program yazılım tabanlı gerçekleştirileceği gibi donanım tabanlı da tasarlanabilir.

Genel olarak bilgisayar programcıları için yazılım tarafıyla ilgilenirken sistem geliştiricileri hem yazılım hem de donanım tarafıyla ilgilenirler.

Bu ders bir yazılım dersi olduğu için program denildiğinde belirli bir dile dayanılarak geliştirilmiş yazılım parçası akla gelecektir.

Yazılım bir veya birden çok programın bir araya gelmesinden oluşan bir program kümesidir, bir sistemdir. Özellikle büyük ölçekli yazılımlar birçok programın ve belgelerin bir araya gelmesi ile oluşur.

Program kodu, bir işin yapılması için algoritma ifadeyi gösteren programın herhangi bir programlama diline dayanılarak ilgili dilin deyimleri fonksiyonları ve özellikleri ile elde edilmiş program parçasıdır. Program kodu yalnızca bir deyimden oluşacağı gibi programın tamamını da kapsayabilir.

Program kodlarının yürütülmesi anında üzerinde işlem yapılacak parametreler **veri** olarak adlandırılır.

Yazılımın en temel ve önemli parçaları; veriler ve verilerin bellekte saklanmasıdır. Çünkü üzerinde işlem yapılacak verilerin bilgisayar belleğinde saklanma şekli ve düzeni onları kullanacak programların tasarımını/algoritmik ifadesini oldukça etkiler. Öyle ki tasarlanması zor görülen bir program verileri saklanma şekli ve düzeni değiştirildiğinde kolayca gerçekleştirilebilir hale gelmektedir.

Verilerin bellek üzerinde saklanma şekli **veri yapısı(data structure)**, ilişkisel düzeni ise **veri modeli(data model)** olarak bilinir. Bu yapıları kullanarak belirli bir işi yapan programın algoritmik şekli ise **Algoritma** olarak adlandırılır.

Dr. Öğr. Üyesi Fatma AKALIN

Veri Yapısı (Data Structure)

Veri yapısı verinin veya bilginin bellekte tutulma şeklini/düzenini gösterir. Genel olarak tüm programlama dillerinde tam sayı, kesirli sayı karakter ve sözcük tanımlanması için kullanılan temel veri yapıları vardır. Programcı bu veri yapılarını bolca kullanır. Aynı zamanda tasarlanacak program temel veri yapısı dışında farklı veri yapılarına ihtiyaç duyuyorsa bunlar kullanılan programlama dilinin el verdiği ölçüde tanımlanmalıdır Bu amaçla temel veri yapıları dışında yeni veri yapılarının tanımlanması için programlama dillerinde birçok özellik mevcuttur. Örneğin; C programlama dilinde struct ve union deyimleri kullanılarak yeni veri yapıları oluşturulmaktadır.

Veri Modeli (Data Model)

Veri modeli verilerin birbirleriyle ilişkisel veya sırasal durumunu gösterir. Bilgisayar ortamında uygulanacak tüm matematik ve mühendislik problemleri bir veri modeline yaklaştırılarak veya yeni veri modelleri tanımlaması yapılarak çözülebilmektedir. Örneğin ağaç ve graf veri modeli yaygın olarak kullanılan iki farklı veri modelidir.

Özellikle uygun veri modeli ile çözüme gidilemeyen problemler çoğu zaman çözümsüz kalmakta veya tasarımcı tarafından bellek yetmiyor, bilgisayarın hızı yetmiyor gibi sebeplerle yarım kalmaktadır.

Uygulamada her problem doğası gereği en uygun bir veri modeline sahiptir. Bu modelin yazılım geliştiricisi veya programcı tarafından görülebilmesi gerekir. Daha sonra bu veri modeline dayanarak algoritmik ifadeler ortaya koyulmalıdır. Tabii ki yeni veri modelleri de yeni veri yapılarının tanımlanmasını gerektirecektir.

Günümüze kadar liste, ağaç, graf, sonlu durum makinesi gibi birçok veri modeli geliştirilmiş olup bu veri modellerine ait birçok fonksiyon tanımlanmıştır.

Algoritma / Algorithm

Algoritma, belirli bir görevi var olan ya da sonradan tanımlanan veri modeline dayandırılarak adım adım ortaya koymak ve bunu bilgisayar ortamında herhangi bir programlama dili ile kodlamaktır. Bir başka açıdan algoritma, belirli bir işin veri yapılarıyla algoritmik ifadesini ortaya koymaktır.

Bir program tasarlanması ne kadar güç görünse de gerekli veri modeli ve veri yapısı belirlendiği ve algoritmik ifadesi ortaya konulduğu takdirde kolayca kodlanabilir. Böylelikle programı tasarlamakla geliştirmek oldukça kolay olur. Program tasarlanırken kullanılacak veri yapısı ve algoritma doğrudan uygulamaya bağlıdır.

Bazı uygulamalar için programlama dilinin hali hazırda sahip olduğu veri modelleri yeterliyken bazı uygulamalar için programcı var olan veri modellerini de kullanarak yeni veri modelleri tanımlar ve programın algoritmik ifadesini bunlara dayandırarak geliştirir.

Örneğin dizi uygulaması gerektiren bir program tasarımında dizi yapısı tanımlanmasına gerek yoktur Çünkü C programlama dilinde dizi tanımlaması doğrudan değişken bildirimi yapılırken gerçekleştirilebilir. Ancak uygulama bir ağaç modeline ihtiyaç duyuyorsa ağaç yapısının kurulması için yeni bir veri yapısına gerek duyulur. Eğer programlama dili ağaç veri modelini doğrudan destekleseydi yeni bir veri yapısı tanımlanmasına ihtiyaç duyulmayacaktı Örneğin LISP dili doğrudan ağaç yapısına desteklemektedir Dolayısıyla LISP kullanılıyor olsaydı ağaç veri yapısının tanımlanmasına gerek olmayacaktı.

Algoritma, Belirli bir işi problemin sonucunu elde etmek için art arda uygulanacak adımlar ve koşullar ortaya koyar. Kesin olarak bu adımlar ilgili koşullar altında adım adım izlendiğinde bir sonraki sonuca ulaşır. Bilgisayar dünyasında bir yazılım geliştirilirken birçok algoritmaya ihtiyaç duyulur. Arama algoritması, sıralama algoritması, bir matematiksel modelin çözümlenmesi algoritması gibi birçok algoritma türü vardır. Bu nedenle uygulama geliştirilirken bunların bir veya birkaçı her zaman kullanılır. Üstelik bir algoritma türü içinde birden çok çözüm ortaya atılmıştır. Yani diğer bir deyişle aynı işi yapan birden çok algoritma vardır. Hangi çözümün en uygun olduğu kararı doğrudan uygulamaya ve programcının birikimine bağlıdır. Örneğin 1 milyon tane kaydın olduğu bir veri tabanında bir bilginin aranmasını bir milyon tane sorgulama ile yapan arama algoritması olduğu gibi on tane sorgulama ile yapan arama algoritması da vardır.

Program Çalışma Hızı ve Büyük O / Running Time and Big O

Programın çalışma hızı karmaşıklık ifadesi ile gösterilir. Bu kavram genellikle zaman birimi ile ifade edilmeyip doğrudan işlem adedi veya döngü sayısı ile belirtilir. Çünkü programın çalışma hızındaki zaman miktarı programın üzerinde koştuğu donanıma bağlıdır. Dolayısıyla algoritmaları birbirleriyle karşılaştırmak için zaman miktarını kullanmak gerçekçi olmayacak ve yanılgılara neden olacaktır. Bunun yerine ilgili algoritmanın bilgisayar donanımından bağımsız olarak kaç adet işlem veya döngüyle gerçekleştirilebileceği hesaplanmalıdır. Bu durum algoritma analizinin bir konusudur.

Algoritmelerde karmaşıklık önemli bir kavramdır ve programcıya iki önemli bilgi verir.

İlki programın sonuca ulaşması için yaklaşık olarak kaç çevrim yapılacağı veya toplama çarpma gibi kaç tane aritmetik işleme ihtiyaç duyacağıdır.

İkincisi aynı işi yapan algoritmaların birbirleriyle karşılaştırma olanağına sahip olmasıdır.

Dolayısıyla bir algoritmanın karmaşıklığı hesaplanırsa algoritmanın **çalışma hızı ve bellek ihtiyacının** ne kadar olacağı öngörülebilir.

Ek olarak algoritma karmaşıklığı iki açıdan ele alınır. Birincisi zaman karmaşıklığı diğeri bellek veya alan karmaşıklığıdır. **Zaman karmaşıklığı** algoritmanın sonuca ulaşması için gerekli zaman hakkında tahmini bir bilgi verirken **alan karmaşıklığı** ise algoritmanın ihtiyaç duyacağı bellek miktarı hakkında bilgi vermektedir.

ALGORİTMA ANALİZİ

Algoritma analizi, bir algoritmanın performansını değerlendirmek için kullanılan matematiksel teknikleri içerir ve bu analiz, bilgisayardan, programlama dilinden, programcıdan ve veri yapısından bağımsız olarak yapılmalıdır. Bu, algoritmanın genel etkinliğini ve verimliliğini evrensel olarak değerlendirebilmek için gereklidir. İşte detaylı açıklama:

Neden Önemlidir?

Bir algoritmanın performansını anlamak için en kritik metrik, algoritmanın girdi büyüklüğüne göre ne kadar iş yaptığını ölçmektir. Bu nedenle, **önemli işlemlerin sayısını** hesaplamak ilk adımdır.

Dr. Öğr. Üyesi Fatma AKALIN

Önemli İşlemler Nelerdir?

Karşılaştırmalar: Sıralama algoritmalarında ana maliyet faktörüdür.

Atamalar: Değişkenlere değer atama işlemleri.

Döngü İterasyonları: Döngülerin kaç kez çalıştığı.

Fonksiyon Çağrılarları: Özellikle rekürsif algoritmalarda önemlidir.

Nasıl Hesaplanır?

Algoritmanın adım adım analiz edilmesiyle yapılır.

Her bir önemli işlem için **girdi büyüklüğüne (n)** bağlı olarak kaç kez tekrarlandığı sayılır.

Dr. Öğr. Üyesi Fatma AKALIN

Örnek 1: Basit Koşul ve Atama

```
if (n < 0):      # maliyet: c1    tekrar: 1
    val = -n    # maliyet: c2    tekrar: 1
else:
    val = n     # maliyet: c3    tekrar: 1
```

Analiz:

if (n < 0) koşulu: Sabit zamanda çalışır → c1, 1 kez çalışır.

absval = -n: Sadece n < 0 olduğunda çalışır → c2, 1 kez çalışır.

absval = n: Sadece n ≥ 0 olduğunda çalışır → c3, 1 kez çalışır.

Toplam Çalışma Süresi: Bu koşul yapısı sabit sayıda işlem yaptığı için, toplam karmaşıklık: $O(1)$

Örnek 2:Basit Atamalar

```
count = count + 1  # maliyet: c1    tekrar: 1  
sum = sum + count  # maliyet: c2    tekrar: 1
```

Analiz: Her iki satır da sabit zamanda çalışır.Her biri yalnızca 1 kez çalışır.

Toplam Çalışma Süresi: $O(1)$

Örnek 3:İkili Döngü

```
for i in range(n): # dış döngü maliyeti: c1 tekrar: n
for j in range(n): # iç döngü maliyeti: c2 tekrar: n
print(i, j) # işlem maliyeti: c3 tekrar: n * n
```

Analiz: Dış Döngü: i değişkeni 0'dan n-1'e kadar gider, yani n kez çalışır.İç Döngü: Her i değeri için j değişkeni 0'dan n-1'e kadar gider. Dolayısıyla her dış döngü için n kez çalışır.print(i, j): İç döngüde her iterasyonda çalışır. Toplamda $n * n = n^2$ kez çalışır.

Toplam Çalışma Süresi: İç içe iki döngü olduğu için: $O(n^2)$

Örnek 4: Tek Döngü

```
for i in range(n):      # maliyet: c1 tekrar: n
    print(i)           # maliyet: c2 tekrar: n
```

Analiz: Döngü, i değişkeni 0'dan n-1'e kadar gider, yani n kez çalışır. print(i) her döngüde çalışır, toplamda n kez çalışır.
Toplam Çalışma Süresi: $O(n)$

Örnek 5: Logaritmik Döngü

```
i = 1
while i < n:      # maliyet: c1 tekrar: log(n)
print(i)          # maliyet: c2 tekrar: log(n)
i = i * 2         # maliyet: c3 tekrar: log(n)
```

Analiz: i her adımda iki katına çıkıyor (1, 2, 4, 8, ...). Bu durumda döngü $\log_2(n)$ kez çalışır. Çünkü i değeri en fazla n olana kadar iki katına çıkmaya devam eder.

Toplam Çalışma Süresi: $O(\log n)$

Örnek 6: Üçlü Döngü

```
for i in range(n): # dış döngü tekrar: n
    for j in range(n): # orta döngü tekrar: n
        for k in range(n): # iç döngü tekrar: n
            print(i, j, k) # işlem tekrar: n * n * n
```

Analiz: Dış Döngü: n kez çalışır. **Orta Döngü:** Her dış döngü için n kez çalışır. **İç Döngü:** Her orta döngü için n kez çalışır. **Toplam işlem sayısı:** $n \times n \times n = n^3$

Toplam Çalışma Süresi: $O(n^3)$

Verimi Büyüme Fonksiyonu ile İfade Etme

Büyüme Fonksiyonu Nedir?

Algoritmanın girdi büyüklüğüne göre nasıl büyüdüğünü gösterir.

Asimptotik analiz kullanılarak ifade edilir.

Big-O Notasyonu en yaygın kullanılanıdır.

Big-O Notasyonu, En kötü durum karmaşıklığını ifade eder.

Big- Ω (Omega) notasyonu, algoritmanın **en iyi durum** karmaşıklığını ifade eder.

Big- Θ (Theta) notasyonu, bir algoritmanın **ortalama durum** karmaşıklığını ifade eder.

Yaygın Big-O Notasyonları:

$O(1)$: Sabit zaman, girdi büyüklüğünden bağımsız.

$O(\log n)$: Logaritmik zaman, genellikle böl ve fethet algoritmalarında görülür.

$O(n)$: Doğrusal zaman, tüm elemanlar bir kez işlenir.

$O(n \log n)$: Birleşim zamanlı algoritmalar (ör. Merge Sort).

$O(n^2)$: İkili döngülerde sıkça görülür (ör. Bubble Sort).

$O(2^n)$: Üstel zaman, genellikle rekürsif algoritmalarda.

$O(n!)$: Faktöriyel zaman, olasılık tabanlı algoritmalar (ör. Permütasyonlar).

Nasıl İfade Edilir?

En yüksek dereceli terim dışındaki terimler atılır.

Sabit katsayılar göz ardı edilir.

Örnek:

$T(n) = 3n^2 + 5n + 2$ ise,

En yüksek dereceli terim: n^2

Sabitler atılır.

Karmaşıklık: $O(n^2)$

Örnek 1: Doğrusal Arama

```
def linear_search(arr, x):  
    for i in range(len(arr)):  
        if arr[i] == x:  
            return i  
    return -1
```

AKALIN

Dr. Öğr. Üyesi Fatma AKALIN

En kötü durumda tüm elemanları tek tek kontrol eder.

İşlem sayısı: n (girdi büyüklüğü)

Karmaşıklık: $O(n)$

En iyi durum (Best Case): Eleman ilk pozisyonda $\rightarrow O(1)$

Karmaşıklık: $O(1)$

Karmaşıklık, algoritmanın **verimliliğini** değerlendirmede önemli bir rol oynar. Özellikle sıklıkla karşılaşılan **arama** ve **sıralama** işlemlerinde karmaşıklığın hesaplanması temel bir işlemdir.

Bu derste, sıralama algoritmaları arasından **Bubble Sort, Selection Sort ve Insertion Sort** algoritmaları incelenecektir.

Dr. Öğr. Üy

Bubble Sort

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        swapped = False  
        for j in range(0, n-i-1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j] # Takas işlemi  
                swapped = True  
        if not swapped: # Takas olmadıysa dizi sıralıdır  
            break
```

Algoritma:

Bubble Sort, komşu elemanları karşılaştırarak büyük olanı sağa kaydırır ve bu işlemi dizi tamamen sıralanana kadar tekrarlar.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j] # Takas işlemi
                swapped = True
        if not swapped: # Takas olmadıysa dizi sıralıdır
            break
```

En İyi Durum:Dizi zaten sıralı ise (örneğin: [1, 2, 3, 4, 5]):

İç döngü her elemanı sadece bir kez kontrol eder ve swapped değişkeni False kalır.Bu durumda dış döngü yalnızca 1 kez çalışır ve break ile döngü sonlanır.En iyi durum karmaşıklığı: $O(n)$ Çünkü sadece n elemanı kontrol eder ve takas yapmaz.

En Kötü Durum:Dizi ters sıralı ise (örneğin: [5, 4, 3, 2, 1]):Her eleman yer değiştirmek zorundadır ve en büyük eleman her turda sona gider.İç ve dış döngü toplamda yaklaşık $n * (n-1) / 2$ kez çalışır.En kötü durum karmaşıklığı: $O(n^2)$ Çünkü her eleman için diğer tüm elemanlarla karşılaştırma yapılır.

Selection Sort

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        min_index = i  
        for j in range(i+1, n):  
            if arr[j] < arr[min_index]:  
                min_index = j  
        arr[i], arr[min_index] = arr[min_index], arr[i] # Takas işlemi
```

Algoritma:

Selection Sort, dizideki en küçük elemanı bulup en baştaki elemanla yer değiştirir ve bu işlemi her turda sıralı kısmı genişleterek tekrarlar.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i+1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i] # Takas işlemi
```

En İyi Durum:Dizi zaten sıralı ise (örneğin: [1, 2, 3, 4, 5]):Her turda minimum eleman zaten yerindedir ama algoritma bunu kontrol etmeye devam eder.Tüm elemanlar karşılaştırılır ve takas işlemi yapılır.En iyi durum karmaşıklığı: $O(n^2)$ Çünkü algoritma takas yapmasa bile tüm karşılaştırmaları yapar.

En Kötü Durum:Dizi ters sıralı ise (örneğin: [5, 4, 3, 2, 1]):Algoritma her turda minimum elemanı bulmak için tüm diziyi tarar ve yer değiştirme yapar.İç ve dış döngü toplamda yaklaşık $n * (n-1) / 2$ kez çalışır.En kötü durum karmaşıklığı: $O(n^2)$ Çünkü her eleman için diğer tüm elemanlarla karşılaştırma yapılır.

Insertion Sort

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key
```

Algoritma:

Insertion Sort, diziyi soldan sağa tarar ve her elemanı doğru pozisyonuna yerleştirerek sıralar. Yeni eleman eklerken, önceki elemanlarla karşılaştırmalar yaparak doğru yeri bulur ve ekleme yapar.

Dr. Öğr. Üyes

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key
```

En İyi Durum:Dizi zaten sıralı ise (örneğin: [1, 2, 3, 4, 5]):İç döngü hiç çalışmaz çünkü $arr[j] > key$ koşulu hiçbir zaman sağlanmaz.Her eleman yalnızca bir kez kontrol edilir ve yerinde kalır.En iyi durum karmaşıklığı: $O(n)$ Çünkü sadece n kez karşılaştırma yapılır ve yer değiştirme olmaz.

En Kötü Durum:Dizi ters sıralı ise (örneğin: [5, 4, 3, 2, 1]):Her eleman eklenirken, kendinden önceki tüm elemanlarla karşılaştırılır ve kaydırılır.Örneğin, 1 sayısını en başa getirmek için tüm elemanlar sağa kaydırılır.En kötü durum karmaşıklığı: $O(n^2)$ Çünkü her eleman, öncesindeki tüm elemanlarla karşılaştırılır ve kaydırılır.

Sizler ile paylaşılan sıralama algoritmaları haricinde bilgisayar bilimi ve yazılım mühendisliği alanlarında **Quick Sort** ve **Merge Sort** gibi sıralama algoritmalarını öğrenmek çok değerli ve faydalıdır. **Algoritma tasarımı** hakkında daha derinlemesine bilgi edinmenize yardımcı olur.

Quick Sort ve Merge Sort öğrenmek, bilgisayar bilimi ve yazılım geliştirme için temel bir beceridir. Bu algoritmaların **verimliliği**, **karmaşıklığı** ve **uygulama alanları** hakkında bilgi sahibi olmak, daha verimli yazılım çözümleri geliştirebilmenizi sağlar ve algoritmaların nasıl çalıştığını kavramanızı sağlar.

Bununla birlikte Veri Yapıları dersinde Algoritma analizine daha fazla girmeyi düşünmüyorum.

Şunu unutmayalım ki,
Veri yapıları, algoritmaların verileri nasıl saklayıp organize ettiğini belirlerken, algoritma karmaşıklığı, bu verilerin işlenme süresini (zaman karmaşıklığı) ve bellekteki yerini (alan karmaşıklığı) analiz eder.
Veri yapıları ve algoritma karmaşıklığı arasındaki ilişkiyi anlamak, yazılım geliştirme ve problem çözme becerilerinizi güçlendirecektir.

Arama algoritmaları arasında **Linear Search**
(Doğrudan Arama), **Binary Search (İkili Arama)**,
algoritmalarını inceleyelim.

Linear Search (Doğrudan Arama)

```
def linear_search(arr, x):  
    for i in range(len(arr)):  
        if arr[i] == x:  
            return i  
    return -1
```

Algoritma:

Linear Search, aranan öğeyi dizinin her elemanı ile tek tek karşılaştırarak bulmaya çalışır. Eğer öğe bulunursa, elemanın indeksini döndürür, aksi takdirde -1 döndürür.


```
def linear_search(arr, x):  
    for i in range(len(arr)):  
        if arr[i] == x:  
            return i  
    return -1
```

En İyi Durum: Aranan öğe dizinin ilk elemanıdır (örneğin: [3, 5, 7, 9] ve aranan 3): İlk karşılaştırmada öğe bulunur ve algoritma hemen durur. En iyi durum karmaşıklığı: $O(1)$ Çünkü sadece bir kez karşılaştırma yapılır ve arama bitirilir.

En Kötü Durum: Aranan öğe dizinin son elemanıdır veya dizide bulunmaz (örneğin: [3, 5, 7, 9] ve aranan 10): Algoritma tüm diziyi kontrol eder. En kötü durum karmaşıklığı: $O(n)$ Çünkü her elemanla karşılaştırma yapılır.

Binary Search (İkili Arama)

Algoritma: Binary Search, sıralı bir dizide arama yaparken, dizinin ortasındaki öğeyi kontrol eder. Eğer aranan öğe orta öğeden küçükse, dizinin sol kısmını, büyükse sağ kısmını arar. Bu işlem, öğe bulunana kadar devam eder.

```
def binary_search(arr, x):  
    left = 0  
    right = len(arr) - 1  
    while left <= right:  
        mid = (left + right) // 2  
        if arr[mid] == x:  
            return mid  
        elif arr[mid] < x:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1
```

Dr. Öğr. Üyesi F.

En İyi Durum:Aranan öğe dizinin ortasında bulunur (örneğin: [1, 3, 5, 7, 9] ve aranan 5):İlk karşılaştırmada öğe bulunur ve algoritma hemen durur.En iyi durum karmaşıklığı: $O(1)$ Çünkü sadece bir kez karşılaştırma yapılır.

```
def binary_search(arr, x):
    left = 0
    right = len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

En Kötü Durum:Aranan öğe dizinin en sonunda yer alır veya bulunmaz (örneğin: [1, 3, 5, 7, 9] ve aranan 7):Algoritma her seferinde dizinin yarısını eler ve sonunda öğeyi bulur veya diziyi tamamen tarar.En kötü durum karmaşıklığı: $O(\log n)$ Çünkü her adımda dizinin boyutunu yarıya indirir.

D

Algoritma	En İyi Durum	En Kötü Durum
Linear Search	$O(1)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$

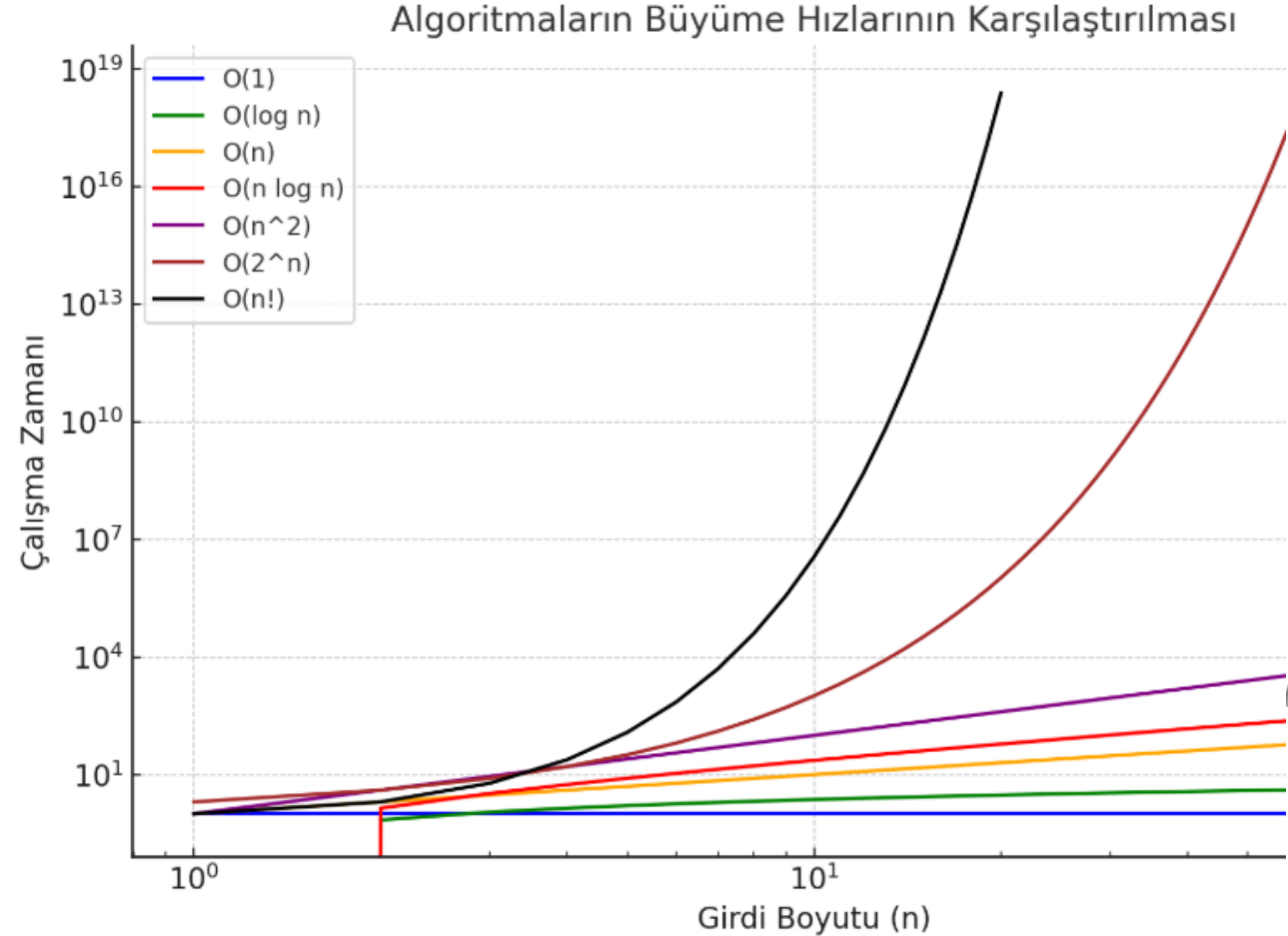
Sıfat

Algoritma	En İyi Durum	En Kötü Durum
Bubble Sort	$O(n)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$

LIN

KIYASLAMA / BENCHMARKING

Kıyaslama aynı işi yapan iki programın veya yazılımın evrensel anlamda örnek veriler üzerinde çalıştırılarak başarımlarını karşılaştırmaktır.



```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Veri hazırlığı
```

```
n = np.arange(1, 101) # Girdi boyutu n
(1'den 100'e kadar)
```

```
O_1 = np.ones_like(n) # O(1)
```

```
O_log_n = np.log(n) # O(log n)
```

```
O_n = n # O(n)
```

```
O_n_log_n = n * np.log(n) # O(n log n)
```

```
O_n2 = n**2 # O(n^2)
```

```
O_2n = 2**n # O(2^n)
```

```
O_n_fact = np.array([np.math.factorial(i) if i
<= 20 else np.inf for i in n]) # O(n!)
```

```
# Grafik oluşturulması
```

```
plt.figure(figsize=(10, 6))
```

```
# Her büyüme hızını grafikte gösterme
```

```
plt.plot(n, O_1, label="O(1)", color='blue')
```

```
plt.plot(n, O_log_n, label="O(log n)", color='green')
```

```
plt.plot(n, O_n, label="O(n)", color='orange')
```

```
plt.plot(n, O_n_log_n, label="O(n log n)", color='red')
```

```
plt.plot(n, O_n2, label="O(n^2)", color='purple')
```

```
plt.plot(n, O_2n, label="O(2^n)", color='brown')
```

```
plt.plot(n, O_n_fact, label="O(n!)", color='black')
```

```
# Grafik özellikleri
```

```
plt.yscale('log') # Y eksenini logaritmik yapıyoruz
```

```
plt.xscale('log') # X eksenini logaritmik yapıyoruz
```

```
plt.xlabel("Girdi Boyutu (n)", fontsize=12)
```

```
plt.ylabel("Çalışma Zamanı", fontsize=12)
```

```
plt.title("Algoritmaların Büyüme Hızlarının Karşılaştırılması",
fontsize=14)
```

```
plt.legend()
```

```
plt.grid(True)
```

```
# Gösterme
```

```
plt.show()
```

- **Grafik Açıklamaları:**

- **$O(1)$:** Sabit zaman, her girdi boyutunda sabit bir süre alır. Y ekseninde düz bir çizgiyle gösterilir.
- **$O(\log n)$:** Logaritmik büyüme. Girdi boyutu arttıkça yavaş bir şekilde artar.
- **$O(n)$:** Doğrusal büyüme. Girdi boyutu ile doğru orantılı olarak artar.
- **$O(n \log n)$:** Logaritmik doğrusal büyüme. Genellikle sıralama algoritmalarında görülür.
- **$O(n^2)$:** Kuadratik büyüme. Girdi boyutunun karesiyle orantılı olarak artar ve büyük veri setlerinde hızla büyür.
- **$O(2^n)$:** Üssel büyüme. Çok hızlı bir şekilde artar ve çok büyük girdi boyutları için pratik olmayabilir.
- **$O(n!)$:** Faktöriyel büyüme. Girdi boyutu çok arttıkça inanılmaz hızlı büyür ve sadece çok küçük girdi boyutları için uygulanabilir.

Programın Bellek Gereksinimi

Bir programın belleğe olan ihtiyacını iki ana kategoriye ayırıyor: **kodun tutulması** ve **kodun çalışması**. Programlar iki farklı bellek gereksinimine sahiptir; biri programın kodunu tutmak (yani çalıştırılabilir dosya veya komutlar), diğeri ise programın çalışması sırasında kullanılan veri belleğidir (örneğin değişkenler, diziler veya diğer veri yapıları).

1.Kodun tutulması:

1. Programın çalıştırılması için gerekli olan kod (yazılımsal komutlar ve işlevler), bellekte bir alan kaplar. Bu alan genellikle küçük olabilir çünkü çoğu programda kod, bellekte sabit bir boyutta tutulur. Örneğin, bazı basit programlar küçük bir kod yapısına sahip olabilir ve dolayısıyla bellek kullanımı az olur.

2.Kodun çalışması:

1. Program çalışmaya başladığında, kod yalnızca programın komutlarını içerir. Ancak asıl bellek kullanımı, program çalışırken veri işleme sırasında ortaya çıkar.

Örnek senaryo:

Diyelim ki **kodu kısa** olan bir program çalıştırılıyor ama çok büyük veri kümesi üzerinde işlem yapıyor. Bu durumda, kod çok küçük olsa da veri kümesi çok büyük olduğu için bellek gereksinimi artar. Örneğin, büyük bir dosyayı okuyan veya büyük sayıda hesaplama yapan bir program, kodunun uzunluğu ile değil, işlediği verinin büyüklüğü ile daha fazla bellek kullanır.

Diğer taraftan, **kodu uzun** olan bir program çalıştırılabilir. Bu programın kodu büyük olsa da verisi küçükse, bellek gereksinimi daha düşük olabilir. Örneğin, büyük bir oyun veya uygulama yazılımı, çok sayıda komut içeriyor olabilir ancak işlediği veri çok küçükse (belirli bir seviyede veri veya kullanıcı etkileşimi ile sınırlıysa) belleği fazla tüketmeyebilir.

Bu bağlamda, bellek kullanımı programın kodunun uzunluğuna değil, büyük ölçüde **programın çalışması sırasında işlediği veriye** bağlıdır. Bu ifade aynı zamanda **programın yürütme zamanı bağıntısı** olarak tanımlanmaktadır.

Bir algoritma 8 elemanlı bir problemi 1 saniyede sonuçlandırıyorrsa 64 elemanlı bir problem için ne kadar zaman gerekir.

Algoritmanın mertebesi:

$O(1) \rightarrow T(n) = 1$ saniye

$O(\log_2 n) \rightarrow T(n) = 2$ saniye

$O(n) \rightarrow T(n) = 8$ saniye

$O(n * \log_2 n) \rightarrow T(n) = 16$ saniye

$O(n^2) \rightarrow T(n) = 64$ saniye

$O(n^3) \rightarrow T(n) = 512$ saniye

$O(2^n) \rightarrow T(n) = 10^{16}$ saniye

ÖZETLE

Yürütme zamanı ve zaman karmaşıklığı, algoritmaların performansını analiz etmek için kullanılan iki farklı kavramdır. Aralarındaki farklar şu şekildedir.

1. Yürütme Zamanı (Execution Time)

Yürütme zamanı, bir algoritmanın bir veri kümesi üzerinde çalışarak tamamlanması için geçen gerçek **zamanı** ifade eder. Bu süre, algoritmanın uygulama sırasında gerçekten geçen süreyi ölçer ve çeşitli dış etmenlere bağlıdır:

Girdi boyutu: Veri setinin büyüklüğü, yürütme zamanını doğrudan etkiler.

İşlemci hızı: Bilgisayarın işlemci gücü, yürütme zamanını etkiler.

Bellek kullanımı: Algoritmanın belleği nasıl kullandığı da zaman üzerinde etkili olabilir.

Sistem koşulları: Diğer programların çalışması, sistem yükü gibi faktörler de yürütme zamanını etkileyebilir.

Yürütme zamanı genellikle **gerçek ölçülen zamandır** ve belirli bir algoritma ile belirli bir veri kümesi için yapılan testler sonucunda elde edilir.

2. Zaman Karmaşıklığı (Time Complexity)

Zaman karmaşıklığı, bir algoritmanın **performansını** veya **işlem süresini**, verilen **girdi boyutuna** (genellikle n olarak ifade edilir) göre tanımlar. Yani, algoritmanın giriş verisinin büyüklüğü arttıkça, algoritmanın çalışma süresindeki artışı belirleyen bir ölçüttür.

Bu, algoritmanın performansını genel bir şekilde analiz etmeye yarar ve genellikle **Big O notasyonu** ile ifade edilir. Zaman karmaşıklığı, genellikle şu amaçlarla kullanılır:

- Algoritmanın **en kötü durum** performansını anlamak.
- Büyük veri setleriyle** çalışırken algoritmanın nasıl performans göstereceğini tahmin etmek.
- Farklı algoritmaların **teorik karşılaştırması**.

Zaman karmaşıklığı, dış faktörlerden bağımsızdır ve daha çok algoritmanın yapısal özelliklerine dayanır. Örneğin, $O(n)$, $O(n^2)$, $O(\log n)$ gibi ifadeler, bir algoritmanın performansının girdi boyutuna göre nasıl büyüdüğünü tanımlar.

Farklar:

Yürütme Zamanı: Gerçek dünyada geçirilen zaman (saniye, milisaniye gibi birimler ile ölçülür).

Zaman Karmaşıklığı: Algoritmanın teorik performansı, girdi boyutuna göre nasıl değiştiğini gösterir (genellikle Big O notasyonu ile ifade edilir).

Diyelim ki, bir algoritmanın zaman karmaşıklığı $O(n^2)$. Bu, algoritmanın girdi boyutu n arttıkça çalışma zamanının karesel olarak arttığı anlamına gelir. Ancak gerçek yürütme zamanı, bu algoritmanın ne kadar süre alacağını belirlemek için belirli bir n değeriyle çalıştırıldığında ölçülür. Örneğin, $n = 1000$ olduğunda, algoritma 1 saniye sürebilir, ancak $n = 10,000$ olduğunda 100 saniye sürebilir.

Sonuç olarak, zaman karmaşıklığı, algoritmanın büyüklük açısından nasıl performans gösterdiğini tahmin etmek için kullanılırken, yürütme zamanı, algoritmanın pratikte ne kadar süre çalıştığını gösterir.

SORULAR

- 1-Genel olarak yazılım ile program tanımları nedir ve aralarında ne fark vardır? Örnek vererek açıklayınız.
- 2- Veri yapısı tanımını eğitimde ilköğretim düzeyinde olan bir çocuğun anlayabileceği kadar yalın ve açık bir şekilde yapınız. Veri modelinin veri yapısından farkı nedir Açıklayınız.
- 3- Aynı veri modeli için arama ekleme vs. gibi aynı işlemi yapacak birden çok algoritma geliştirilmektedir. Niçin bu kadar çok algoritma geliştirilmektedir?

4- Programın yürütme zamanı bağıntısı ne demektir? Hangi parametrelere göre hesaplanmaktadır?

5- Bir programın gereksinim duyacağı bellek alanı nasıl hesaplanabilir? Bellek alanı karmaşıklığı ne demektir?

6- Yürütme zamanı ile zaman karmaşıklığı ifadelerinin anlamları arasındaki fark nedir?

KAYNAKÇA

1-Veri Yapıları ve Algoritmalar, 12. Baskı, Rifat Çölkesen

2-Nejat Yumuşak, fatih Adak Veri C++ ile veri yapıları, 3.baskı

3-C++ ile programlama, Dokuzuncu baskıdan çeviri, Palme Yayınları, Pau Deitel,Harvey Deitel, Çeviri Editörü Cemil Öz

4-Veri Yapıları, Hakan Kutucu

5-<https://chatgpt.com/>